DS4001-25SP-HW3：贝叶斯网络

黄思睿　PB23151782

2025 年 5 月 30 日

1　马尔可夫链 [24%]

1.1　回答问题 [20%]

1.1.1　手动计算 [4%]

1. 天气状态为三类：晴天 q_1、阴天 q_2、下雨 q_3。

   该序列可表示为：$\{q_2, q_3, q_1, q_1, q_1, q_2, q_3, q_3\}$

   序列概率计算如下：

$$P(\text{序列}) = \pi_2 \cdot a_{23} \cdot a_{31} \cdot a_{11} \cdot a_{11} \cdot a_{12} \cdot a_{23} \cdot a_{33}$$
$$= 0.5 \cdot 0.2 \cdot 0.3 \cdot 0.8 \cdot 0.8 \cdot 0.1 \cdot 0.2 \cdot 0.4 = 1.536 \times 10^{-5}$$

2. 从初始状态就是下雨开始，连续四天都保持下雨状态的概率为：

$$P(\text{连续四天下雨}) = a_{33} \cdot a_{33} \cdot a_{33} = 0.4 \cdot 0.4 \cdot 0.4 = 0.064 = 6.4 \times 10^{-2}$$

3. 该问题可建模为几何分布，其期望为：

$$\text{期望下雨天数} = \frac{1}{1 - a_{33}} = \frac{1}{1 - 0.4} = \frac{1}{0.6} \approx 1.67\text{天}$$

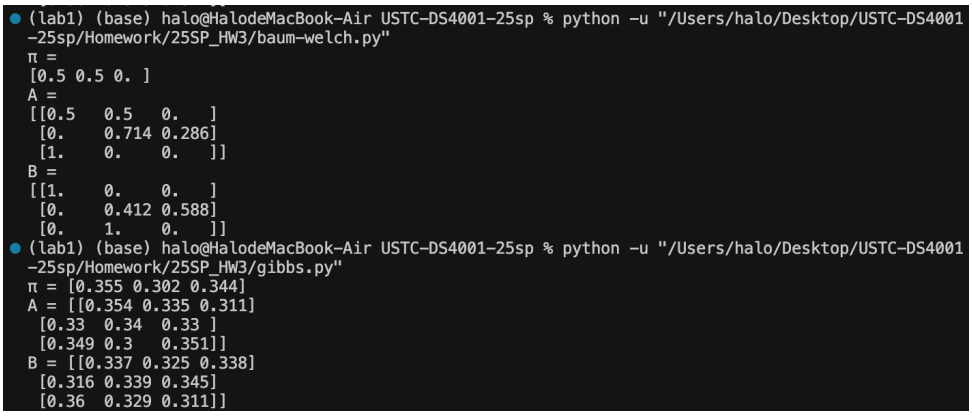
1.1.2　代码填空与参数估计 [16%]

```
alpha[0] = alpha[0] = self.pi * self.B[:, seq[0]]

alpha[t] = np.dot(alpha[t-1], self.A) * self.B[:, seq[t]]

beta[t] = beta[t] = np.dot(self.A * self.B[:, seq[t+1]], beta[t+1])
```

图 1: Bw 运行结果

1.1.3 运行结果对比 [4%]

通过多次运行 Baum-Welch 算法和 Gibbs 采样 (设置相同的随机种子), 观察到两种方法在估计 HMM 参数时存在显著差异:



```
(lab1) (base) halo@HalodeMacBook-Air USTC-DS4001-25sp % python -u "/Users/halo/Desktop/USTC-DS4001
-25sp/Homework/25SP_HW3/baum-welch.py"
π =
[0.5 0.5 0. ]
A =
[[0.5   0.5   0.   ]
 [0.    0.714 0.286]
 [1.    0.    0.   ]]
B =
[[1.    0.    0.   ]
 [0.    0.412 0.588]
 [0.    1.    0.   ]]
(lab1) (base) halo@HalodeMacBook-Air USTC-DS4001-25sp % python -u "/Users/halo/Desktop/USTC-DS4001
-25sp/Homework/25SP_HW3/gibbs.py"
π = [0.355 0.302 0.344]
A = [[0.354 0.335 0.311]
 [0.33  0.34  0.33 ]
 [0.349 0.3   0.351]]
B = [[0.337 0.325 0.338]
 [0.316 0.339 0.345]
 [0.36  0.329 0.311]]
```

图 2: 运行结果对比（上为 bw 算法，下为 Gibbs 采样）

这种差异可以从以下几个方面分析:

算法本质不同: Baum-Welch 算法是 EM 算法在 HMM 上的特例实现，是确定性的最大似然方法，追求找到一组最优参数 θ 使 $P(O|\theta)$ 最大化。而 Gibbs 采样是贝叶斯方法，通过 MCMC 采样来估计参数的后验分布 $P(\theta|O)$，不仅关注点估计。

参数估计特性: Baum-Welch 产生了" 确定性" 很强的模型，许多概率值为 0 或接近 1，形成了明确的决策边界。Gibbs 采样的概率值则集中在 0.3-0.36 之间，更平滑、更均匀，体现了对参数不确定性的建模。

先验信息影响: Baum-Welch 是纯粹的最大似然方法，没有明确的先验。而 Gibbs 采样使用了 Dirichlet 先验（代码中 $\alpha = \beta = \gamma = 1.0$），这促使结果更接近均匀分布，避免出现概率为 0 的情况，增强了泛化能力。

稳定性比较: Baum-Welch 对初始值敏感，容易陷入局部最优，但对同一初始值结果一致。Gibbs 采样即使使用相同初始化，每次运行结果也略有不同，但整体分布模式相似，对初始值较不敏感，表现出更强的稳健性。

2 贝叶斯网络 [64%]

2.1 贝叶斯网络: 推理 [54%]

2.1.1 精确推理 [16%]

```python
    def observe(self, agentX: int, agentY: int, observedDist: float) -> None:
        # BEGIN_YOUR_CODE (our solution is 7 lines of code, but don't worry if you deviate from this)
        numRows, numCols = self.belief.getNumRows(), self.belief.getNumCols()
        # 遍历所有可能的位置
```

```python
        for row in range(numRows):
            for col in range(numCols):
                # 将网格索引转换为实际坐标
                y = util.rowToY(row)
                x = util.colToX(col)

                # 计算当前网格位置与agent之间的真实距离
                trueDist = math.sqrt((x - agentX) ** 2 + (y - agentY) ** 2)

                # 计算观测似然：P(observedDist | position)
                emissionProb = util.pdf(trueDist, Const.SONAR_STD, observedDist)

                # 更新该位置的概率：P(position) * P(observedDist | position)
                currentProb = self.belief.getProb(row, col)
                self.belief.setProb(row, col, currentProb * emissionProb)

        # 归一化，确保所有概率之和为1
        self.belief.normalize()
        # END_YOUR_CODE

    def elapseTime(self) -> None:
        # BEGIN_YOUR_CODE (our solution is 5 lines of code, but don't worry if you deviate from this)
        # 创建一个新的信念分布用于存储更新后的概率
        newBelief = util.Belief(self.belief.getNumRows(), self.belief.getNumCols(), 0)

        # 遍历所有可能的位置对(oldTile, newTile)和对应的转移概率
        for (oldTile, newTile), prob in self.transProb.items():
            # 获取旧位置的概率
            oldProb = self.belief.getProb(oldTile[0], oldTile[1])

            # 将这个概率乘以转移概率，添加到新位置
            newBelief.addProb(newTile[0], newTile[1], oldProb * prob)

        # 归一化新的信念分布
        newBelief.normalize()

        # 更新当前信念
        self.belief = newBelief
        # END_YOUR_CODE
```

上面的 observe 这段代码实现了观测更新。我先遍历了网格中的所有位置，用 rowToY 和 colToX 把行列索引转换为真实坐标，然后计算每个位置到小车的欧氏距离。使用 pdf 函数计算观测似然，也就是在这个位置上看到给定测量距离的概率。最后用贝叶斯公式更新每个位置的概率并归一化。

而下面这个函数实现了时间预测步骤。首先创建一个新的空信念对象，然后遍历所有的转移概率。对于每个转移，取出原位置的概率，乘上转移概率，然后把结果加到新位置上。使用 addProb 是因为一个新位置可能来自多个旧位置。最后归一化并更新信念。

2.1.2 模糊推理 [16%]

```python
    def updateBelief(self) -> None:
        newBelief = util.Belief(self.belief.getNumRows(), self.belief.getNumCols(), 0)

        # BEGIN_YOUR_CODE (our solution is 5 lines of code, but don't worry if you deviate from this)
        # 遍历所有样本和对应权重
        for i, (row, col) in enumerate(self.samples):
```

```
            # 将当前样本的权重添加到对应位置
            newBelief.addProb(row, col, self.weights[i])

        # 归一化信念分布，确保概率总和为1
        newBelief.normalize()

        # 更新当前信念
        self.belief = newBelief
        # END_YOUR_CODE
```

这个函数将样本和权重转换为一个概率分布。我创建一个新的信念对象，然后遍历所有样本，把每个样本的权重添加到对应的位置上。如果多个样本在同一位置，它们的权重会累加。最后归一化并更新信念分布。这是似然加权算法的核心部分，将离散样本转化为连续概率分布。

```

    def elapseTime(self) -> None:
        # BEGIN_YOUR_CODE (our solution is 8 lines of code, but don't worry if you deviate from this)
        # 存储更新后的样本
        newSamples = []

        # 遍历当前所有样本
        for i, (row, col) in enumerate(self.samples):
            oldTile = (row, col)

            # 如果当前位置有转移概率定义
            if oldTile in self.transProbDict:
                # 根据转移概率随机选择下一个位置
                newTile = util.weightedRandomChoice(self.transProbDict[oldTile])
                newSamples.append(newTile)
            else:
                # 如果没有定义转移，保持在原位置
                newSamples.append(oldTile)

        # 更新样本列表
        self.samples = newSamples

        # 更新信念分布
        self.updateBelief()
        # END_YOUR_CODE
```

这段代码移动每个样本，模拟车辆随时间变化的位置。首先遍历所有样本，对每个样本，如果它所在位置有定义转移概率，就根据这些概率随机选择一个新位置；如果没有定义，就保持原位置不变。把所有新样本存入列表，然后更新样本集合和信念分布。这里处理了没有转移定义的情况，确保代码在所有情况下都能正常运行。

2.1.3 粒子滤波 [16%]

```
    def __init__(self, numRows: int, numCols: int):
        self.belief = util.Belief(numRows, numCols)

        # Load the transition probabilities and store them in an integer-valued defaultdict.
        # Use self.transProbDict[oldTile][newTile] to get the probability of transitioning from oldTile to newTile.
        self.transProb = util.loadTransProb()
        self.transProbDict = dict()
        for (oldTile, newTile) in self.transProb:
```

```
            if oldTile not in self.transProbDict:
                self.transProbDict[oldTile] = collections.defaultdict(int)
            self.transProbDict[oldTile][newTile] = self.transProb[(oldTile, newTile)]

        # BEGIN_YOUR_CODE (our solution is 5 lines of code, but don't worry if you deviate from this)
        # 初始化粒子
        self.particles = collections.defaultdict(int)
        # 获取所有有转移概率定义的位置
        valid_positions = list(self.transProbDict.keys())
        if valid_positions:  # 确保存在有效位置
            for _ in range(self.NUM_PARTICLES):
                pos = random.choice(valid_positions)
                self.particles[pos] += 1
        else:  # 如果没有有效位置，回退到随机初始化
            for _ in range(self.NUM_PARTICLES):
                row = random.randint(0, numRows - 1)
                col = random.randint(0, numCols - 1)
                self.particles[(row, col)] += 1
        # END_YOUR_CODE
        self.updateBelief()
```

这段代码初始化粒子滤波器的粒子分布。我选择只在有转移概率定义的位置上放置粒子，这样可以避免后面出现 KeyError。如果找不到有效位置，就退回到随机初始化。用 defaultdict 存储粒子计数很方便，因为它自动处理新位置。这种初始化方法比纯随机更合理，因为粒子只会出现在”合法”的位置上。

```
    def observe(self, agentX: int, agentY: int, observedDist: float) -> None:
        # Reweight the particles
        # BEGIN_YOUR_CODE (our solution is 7 lines of code, but don't worry if you deviate from this)
        # 重加权粒子
        newParticles = collections.defaultdict(float)

        # 遍历所有粒子位置
        for (row, col), count in self.particles.items():
            if count > 0:  # 只处理有粒子的位置
                # 计算粒子位置到agent的真实距离
                x = util.colToX(col)
                y = util.rowToY(row)
                trueDist = math.sqrt((x - agentX) ** 2 + (y - agentY) ** 2)

                # 计算观测似然：P(o_t | x_t^(i))
                likelihood = util.pdf(trueDist, Const.SONAR_STD, observedDist)

                # 更新该位置的权重：w_t^(i)   w_{t-1}^(i) · P(o_t | x_t^(i))
                # 这里w_{t-1}^(i)就是count(旧粒子数量)
                newParticles[(row, col)] = count * likelihood

        # 更新粒子集合
        self.particles = newParticles
        # END_YOUR_CODE
```

这个函数实现了粒子滤波的重加权步骤。根据观测值更新每个粒子的权重。我遍历所有有粒子的位置，计算该位置到车的距离，然后用 pdf 函数计算观测似然。新的权重是原来的粒子数乘以似然值。只处理有粒子的位置可以提高效率。重加权后，代码会自动进行重采样（这部分代码已经提供），从更新后的分布中选择新的粒子集合。

2.1.4 思考题 [6%]

调用位置差异

- **LikelihoodWeighting 类**：
  - 在 `observe()` 方法末尾调用 `self.updateBelief()`
  - 在 `elapseTime()` 方法末尾也调用 `self.updateBelief()`
- **ParticleFilter 类**：
  - 在 `observe()` 方法中重采样后调用 `self.updateBelief()`
  - 但在 `elapseTime()` 方法末尾明确注释不需要调用 `self.updateBelief()`

算法思想上

1. 算法表示方式不同

- **似然加权**：样本和权重是分开维护的。样本代表位置，权重代表该位置的可能性。信念分布需要通过显式计算样本位置和权重的组合来更新。
- **粒子滤波**：粒子本身就是概率分布的离散表示。粒子数量直接反映了位置概率，不需要额外维护权重数组。粒子的分布已经隐含地表示了状态概率分布。

2. 更新机制不同

- **似然加权**：
  - 在 `observe()` 中：更新每个样本的权重，但样本位置不变，需要重新计算信念分布。
  - 在 `elapseTime()` 中：根据转移模型移动样本，同样需要重新计算整体信念分布。
- **粒子滤波**：
  - 在 `observe()` 中：先重加权粒子，然后通过重采样创建新粒子分布，这显著改变了状态表示，需要更新信念。
  - 在 `elapseTime()` 中：只是根据转移模型移动粒子，粒子数量分布已经直接反映了新的状态概率，理论上不需要再次计算信念。

因此，`LikelihoodWeighting` 和 `ParticleFilter` 两种算法在调用 `self.updateBelief()` 的位置和次数上存在显著不同。

2.2 贝叶斯网络：学习 [10%]

- 初始参数：$\pi^{old} = 0.5, \theta_A^{old} = 0.6, \theta_B^{old} = 0.4$

2.2.1 E 步：计算第 1 轮后验概率

对于第 1 轮观测数据（正、正、反、正、正），我们需要计算该数据来自硬币 A 和硬币 B 的后验概率。设 Z 表示选择的硬币，X 表示观测结果，我们要计算 $P(Z=A|X)=\gamma_{1,A}$ 和 $P(Z=B|X)=\gamma_{1,B}$。按贝叶斯定理：

$$P(Z=A|X)=\frac{P(X|Z=A)P(Z=A)}{P(X)} \tag{1}$$

$$P(Z=B|X)=\frac{P(X|Z=B)P(Z=B)}{P(X)} \tag{2}$$

其中：

$$P(X|Z=A)=(\theta_A^{old})^4\times(1-\theta_A^{old})^1=0.6^4\times0.4^1=0.05184 \tag{3}$$

$$P(X|Z=B)=(\theta_B^{old})^4\times(1-\theta_B^{old})^1=0.4^4\times0.6^1=0.01536 \tag{4}$$

$$P(X)=P(X|Z=A)P(Z=A)+P(X|Z=B)P(Z=B) \tag{5}$$

$$=0.05184\times0.5+0.01536\times0.5=0.0336 \tag{6}$$

因此：

$$\gamma_{1,A}=P(Z=A|X)=\frac{0.05184\times0.5}{0.0336}=0.7714 \tag{7}$$

$$\gamma_{1,B}=P(Z=B|X)=\frac{0.01536\times0.5}{0.0336}=0.2286 \tag{8}$$

2.2.2 M 步：更新参数

假设所有轮次的后验概率与第 1 轮相同，即 $\gamma_{2,A}=\gamma_{3,A}=\gamma_{4,A}=\gamma_{1,A}=0.7714$ 和 $\gamma_{2,B}=\gamma_{3,B}=\gamma_{4,B}=\gamma_{1,B}=0.2286$。

更新混合系数 π：

$$\pi^{new}=\frac{\gamma_{1,A}+\gamma_{2,A}+\gamma_{3,A}+\gamma_{4,A}}{4} \tag{9}$$

$$=\frac{0.7714+0.7714+0.7714+0.7714}{4}=0.7714 \tag{10}$$

统计所有轮次的抛掷结果：

- 总共 9 个正面，11 个反面

对于硬币 A，计算 θ_A^{new}：

$$\theta_A^{new}=\frac{\text{使用硬币 A 时正面的期望次数}}{\text{使用硬币 A 的总抛掷次数}} \tag{11}$$

$$=\frac{4\times0.7714+1\times0.7714+3\times0.7714+1\times0.7714}{5\times0.7714\times4} \tag{12}$$

$$=\frac{9\times0.7714}{20\times0.7714}=\frac{9}{20}=0.45 \tag{13}$$

对于硬币 B，计算 θ_B^{new}：

$$\theta_B^{new}=\frac{\text{使用硬币 B 时正面的期望次数}}{\text{使用硬币 B 的总抛掷次数}} \tag{14}$$

$$=\frac{4\times0.2286+1\times0.2286+3\times0.2286+1\times0.2286}{5\times0.2286\times4} \tag{15}$$

$$=\frac{9\times0.2286}{20\times0.2286}=\frac{9}{20}=0.45 \tag{16}$$

因此，更新后的参数为：

$$\pi^{new} = 0.7714 \tag{17}$$

$$\theta_A^{new} = 0.45 \tag{18}$$

$$\theta_B^{new} = 0.45 \tag{19}$$

3 贝叶斯深度学习 [6%]

贝叶斯深度学习 (Bayesian Deep Learning, BDL) 通过将贝叶斯推理与深度学习相结合，为解决传统深度学习的关键问题提供了有力工具。贝叶斯方法的核心优势在于其能够量化不确定性，这恰恰弥补了传统深度学习模型的不足。

3.1 贝叶斯方法解决传统深度学习问题的机制

应对过高置信预测 传统神经网络往往对其预测结果给出过高的置信度，即使在面对模糊或不熟悉的输入时也是如此。贝叶斯神经网络通过学习参数分布而非点估计，能够自然地量化认知不确定性，在缺乏足够证据时降低预测置信度。具体而言，贝叶斯方法通过计算预测的后验分布，提供了预测结果的概率分布，而不仅仅是单一输出值，从而更准确地反映模型的不确定性。

改善分布外数据预测 当传统深度学习模型面对训练分布之外的数据时，通常会产生不可靠的高置信度预测。贝叶斯方法通过对模型参数进行采样，可以检测这种分布偏移。参数分布的变异性在面对分布外数据时会产生更大的预测方差，从而提供了一种内置的异常检测机制。这使得模型能够识别” 不知道的未知”，在面对未见过的数据模式时表现出适当的不确定性。

增强对抗性攻击防御 贝叶斯神经网络对对抗性样本表现出更强的鲁棒性。通过对权重进行边缘化 (marginalization)，预测结果不再依赖于单一参数集，而是多种可能参数的综合结果。对抗样本通常仅对特定参数值有效，而权重的随机性使得攻击者更难找到普适的对抗扰动。Bekasov 等人 (2018) 的研究表明，贝叶斯神经网络在面对 FGSM 和 PGD 等常见对抗攻击时，表现出显著更高的鲁棒性。

3.2 贝叶斯反向传播 (BBP) 应用案例

Blundell 等人 (2015) 在《Weight Uncertainty in Neural Networks》中提出的贝叶斯反向传播 (Bayes By Backprop, BBP) 是 BDL 的典型应用。该方法通过以下机制增强了深度学习模型：

1. **权重分布表示**：BBP 将神经网络的权重表示为概率分布而非单一数值，通常使用高斯分布 $\mathcal{N}(\mu, \sigma^2)$。
2. **变分推断**：使用变分推断近似后验分布 $p(w|D)$，优化目标为最小化真实后验与近似后验间的 KL 散度。
3. **重参数化技巧**：通过 $w = \mu + \sigma \cdot \epsilon, \epsilon \sim \mathcal{N}(0, 1)$ 的重参数化，实现了对随机权重的高效梯度计算。
4. **应用实例**：作者在 MNIST 数据集上展示了 BBP 的优势，不仅在分类准确率上与传统方法相当，更重要的是，BBP 能够提供预测的不确定性估计，并自动实现模型复杂度正则化。

在自动驾驶系统中，BBP 的应用显示出特别重要的价值。Kendall 和 Gal(2017) 的研究表明，贝叶斯神经网络能够区分偶然不确定性(数据固有噪声) 和认知不确定性(模型知识缺乏)，这对于自动驾驶决策至关重要——系统可以根据不确定性的来源采取不同的行动策略，如在认知不确定性高的情况下 (例如遇到从未见过的道路情况) 降低车速或请求人工接管。

3.3 其他贝叶斯深度学习应用

除 BBP 外，贝叶斯深度学习还有多种实现方式：

- **Monte Carlo Dropout**(Gal & Ghahramani, 2016)：证明了 dropout 可以视为变分贝叶斯近似，通过前向传播时多次应用 dropout 可以估计预测不确定性。
- **深度集成** (Deep Ensembles, Lakshminarayanan et al., 2017)：虽然不是严格的贝叶斯方法，但通过集成多个具有不同初始化的神经网络，可以有效估计预测不确定性。
- **变分自编码器** (VAE, Kingma & Welling, 2013)：将贝叶斯推断应用于生成模型，学习数据的潜在空间表示及其不确定性。

贝叶斯深度学习虽然在计算复杂度上高于传统方法，但其提供的不确定性量化和增强的鲁棒性，使其成为构建可靠 AI 系统的重要工具，特别是在医疗诊断、自动驾驶等高风险应用领域。随着算法和硬件的进步，贝叶斯深度学习有望在更广泛的场景中发挥关键作用。

体验反馈 [6%]

(a) [**必做**] 五个半天

(b) [**选做**] TODO